

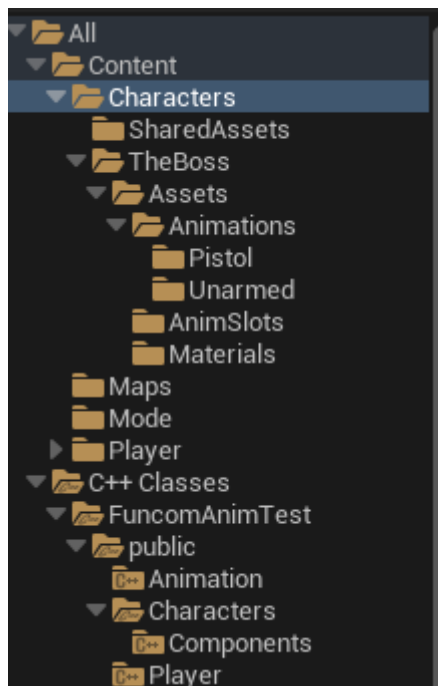
Ricardo Monteiro

Funcom Animation Coding Test Task

Instructions:

- 1 - Make an animation system in Unreal Engine using one main animation blueprint on the skeletal mesh component with one linked anim layer within the main animation blueprint for locomotion.
- 2 - Link the locomotion anim layer instance at runtime by hitting a key to demonstrate multiple locomotion sets working on the same main animation blueprint.
- 3 - Your Locomotion Animations should include walking, possible crouched variations, and jumping/falling, with appropriate transitions between states.
- 4 - Feel free to use any of the sample animations provided by Unreal, as well as any other freely obtained assets (e.g. Mixamo).

Folder Structure:



- **Characters:** This folder contains all the assets related to each character.
- **SharedAssets:** This folder contains the animation layer interface and template animation blueprints.
- **TheBoss:** Contain the playable character assets. This character was extracted from Mixamo and I've added a root bone in blender to it.
- **Assets:** Contains all the assets related to the boss.
- **Animations:** This folder contains all the animation sequences.
- **AnimSlots:** This folder contains all the animation slot files and animation slot data tables.
- **Mode:** It only contains a custom game mode for the test map.
- **Player:** It only contains player related assets, such as the player controller, or camera assets.

C++ Classes:

These follow a similar structure logic to the content folders.

- **Animation:**
 - AnimSlotAsset: This is a UDataAsset that is only used as an ID, such as Proxy Assets, although these have more logic and functionality.
 - AnimTableAsset/LocoAnumTableAsset: These are UDataAssets that contain maps from UAnimSlotAsset to AnimationSequenceBase such as Proxy Tables.

- *FCTBaseAnimInstance*: This is the base animation instance used for all animation blueprints, they support animation slots.
- **Character:**
 - *CharInfo*: This class purpose is to centralize shared information between gameplay and animation. This way animation instances only need to include a weak reference to it, besides all the access to the animation instance already as by default.
 - *FCTBaseCharacter*: This class inherits from ACharacter and is the main class for the playable character in this test.
 - *FCTCharacterMovementComponent*: This class inherits from UCharacterMovementComponent just to override ApplyRootMotionToVelocity.
 - **Components:**
 - InventoryComponent: Just a simple component to exemplify a data table of items to an animation table.

Summary:

In this document I'll go through each requirement and briefly explain how I've achieved it and what my thought process was.

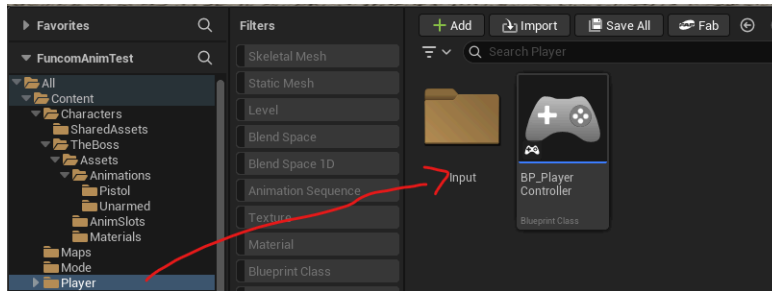
Even though I've used Lyra animations, I've tried to stay clear of it or/and ALS as much as possible, by turning on root motion on said animations and implementing a simple version of Proxy Assets and Plugins, called Animation Slots. Lyra animations contain root motion data but the system does not use them and as such it remains disabled, instead it relies heavily on motion warping, such as orientation and stride warping to match the mesh to the capsule and avoid feet sliding.

Lyra does use animation layer linking to customize state machine states and thus allowing different locomotion animation sequences to switch at runtime. In the current version of Unreal, 5.7 I prefer to use the proxy and chooser tables, but I've avoided including the plugin to be able to demonstrate some code. I've also avoided including motion warping plugins and did just a little bit of procedural manipulation within the animation blueprints.

Requirement 1:

1 - Make an animation system in Unreal Engine using one main animation blueprint on the skeletal mesh component with one linked anim layer within the main animation blueprint for locomotion.

I started a blank project and proceeded to implement everything from scratch, including the camera, player input, map, etc.



For binding input I use a method I personally like, which

is to read the action inputs from the mapping context file associated with the player controller. Then I proceed to find implemented functions with the same name as the actions at possession time.

```
void AFCTPlayerController::SetupInputForPawn(APawn* PossessedPawn)
{
    if (!IsValid(MappingContext))
    {
        return;
    }

    UEnhancedInputComponent* EnhancedInputComponent = Cast<UEnhancedInputComponent>(InputComponent);
    if (!IsValid(EnhancedInputComponent))
    {
        return;
    }

    if (ULocalPlayer* LocalPlayer = Cast<ULocalPlayer>(Player))
    {
        if (UEnhancedInputLocalPlayerSubsystem* InputSystem = LocalPlayer->GetSubsystem<UEnhancedInputLocalPlayerSubsystem>())
        {
            FModifyContextOptions Options;
            InputSystem->AddMappingContext(MappingContext, Priority: 0, Options);

            for (const FEnhancedActionKeyMapping& Mapping : MappingContext->GetMappings())
            {
                // Looks for implemented UFUNCTION with the same name as the action and binds to it
                if (UFunction* Function = PossessedPawn->FindFunction(Mapping.Action.GetFName()))
                {
                    EnhancedInputComponent->BindAction(Mapping.Action, ETriggerEvent::Triggered, PossessedPawn, Mapping.Action.GetFName());
                }
            }
        }
    }
}
```

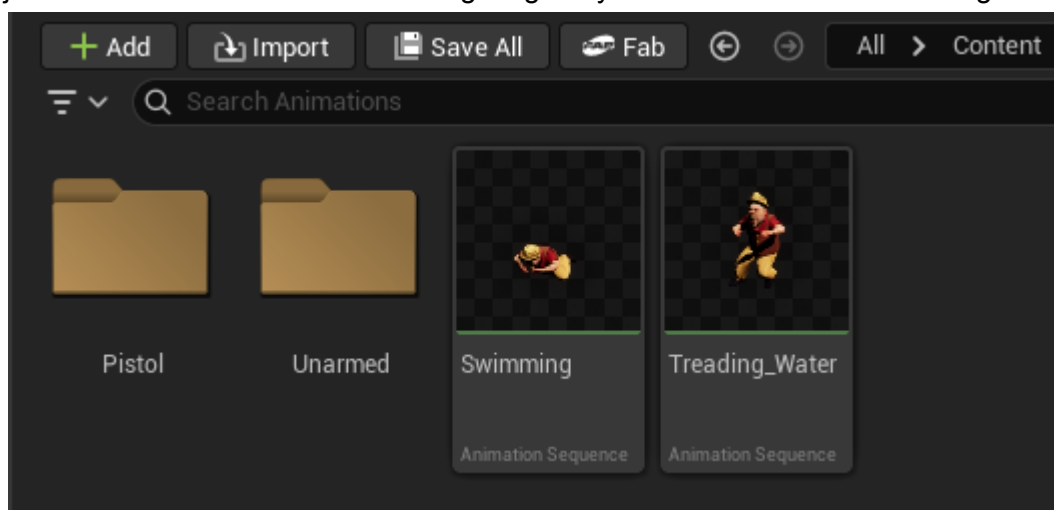
This way I don't have to write references to each action, and I can just make the new action and implement it on the character

```

54 UCLASS()
55 class FUNCOMANIMTEST_API AFCTBaseCharacter : public ACharacter
56 {
57 public:
58
59     const UCharInfo* GetCharInfo() const { return CharInfo; }
60
61 protected:
62     virtual void BeginPlay() override;
63
64     UFUNCTION()
65     void IA_Move(const FInputActionValue& InValue);
66
67     UFUNCTION()
68     void IA_Look(const FInputActionValue& InValue);
69
70     UFUNCTION()
71     void IA_Crouch(const FInputActionValue& InValue);
72
73     UFUNCTION()
74     void IA_ToggleWalk(const FInputActionValue& InValue);
75
76     UFUNCTION()
77     void IA_SwitchEquipment(const FInputActionValue& InValue);
78
79     UFUNCTION()
80     void IA_Jump(const FInputActionValue& InValue);
81
82     UFUNCTION()
83     void IA_Swim(const FInputActionValue& InValue);
84
85 };

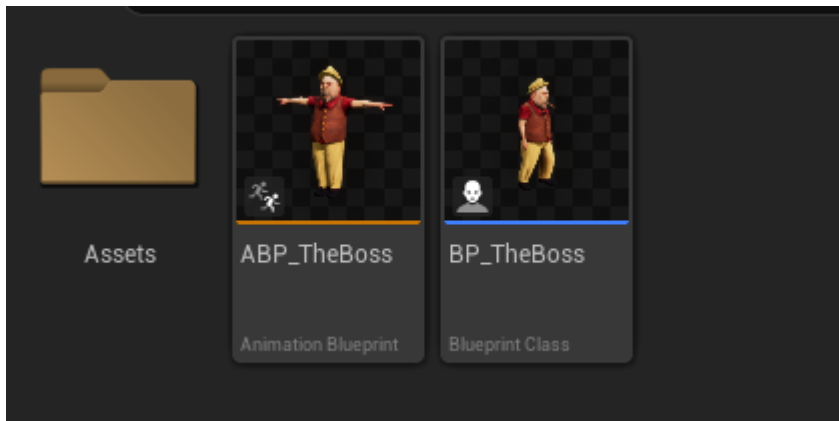
```

To create the playable character I downloaded a Mixamo mesh and processed it in Blender, just to add a root bone so that retargeting of Lyra animations would be straightforward.

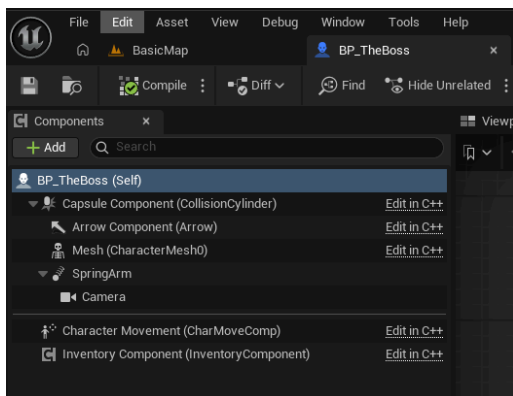


The folder Pistol and Unarmed contain retargeted lyra animations, while the swimming ones were downloaded from mixamo.

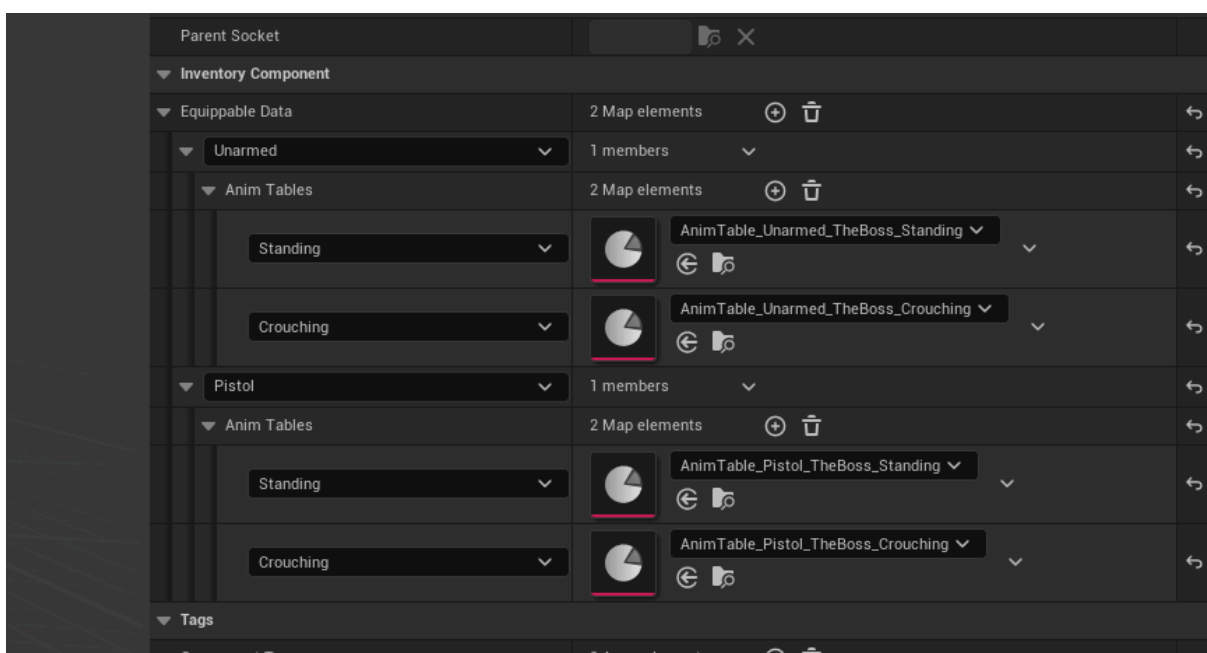
Either way the result is BP_TheBoss and ABP_TheBoss. The boss being the mesh name.



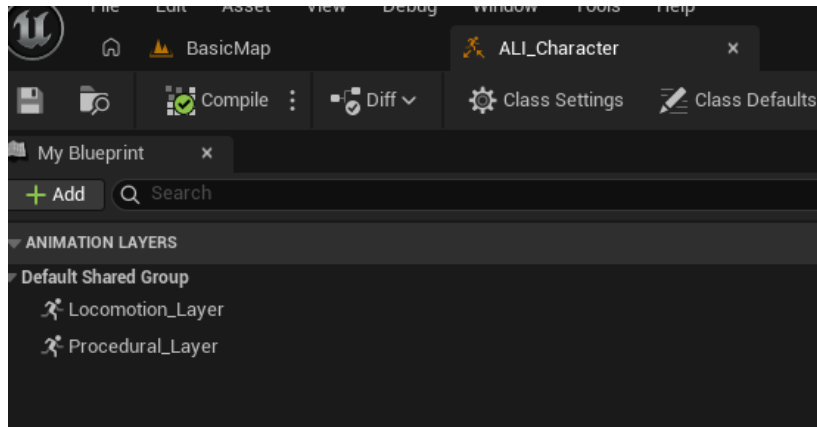
Regarding BP_TheBoss, it's derived from FCTBaseCharacter which is an ACharacter. It contains an added SpringArm and Camera and additionally a custom made InventoryComponent. it's worth mentioning I've made 0 blueprint logic and everything important is C++.



The inventory component only serves to exemplify how each item could be associated to an animation asset table:



I decided to more or less reflect the Lyra approach, with animations being mapped to item/weapon types, because that's how their animations are structured. Then as the requirement states, I've made a main animation blueprint and implemented a simple animation layer interface.

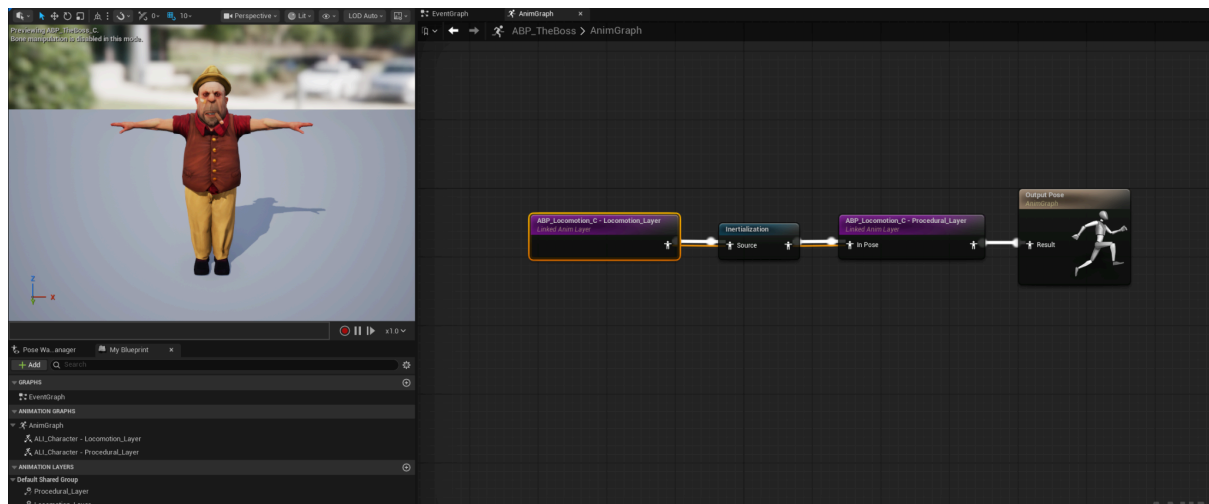


The idea was to not copy Lyra style exactly and move from that towards the proxy/chooser table approach providing a more data friendly approach towards animation. It's worth mentioning that Unreal is moving towards this principle as can be seen with Unreal Animation Framework (previously known as Unreal Anim Next).

This means the animation blueprints are separated from animation asset data, keeping them apart requires less animation layer linking, which might incur performance worries, if able to do by the player frequently, such as switching weapons.

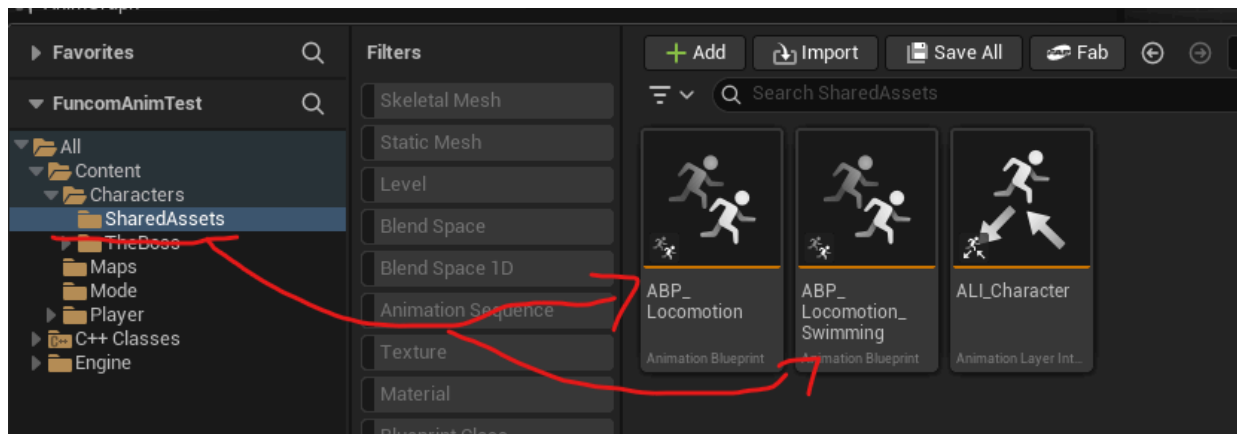
Obviously my implementation is very simple, but with chooser tables we can bind asset selection directly to gameplay data all outside animation blueprints, which means less logic inside them and extra points of parallel work.

With this in mind, the main animation blueprint ABP_TheBoss looks like this:



I only switch linked layers between different blueprints that require different blending/state machine logic. For this test I've implemented grounded locomotion and swimming, with each implementing both locomotion and procedural layer.

Worth mentioning is that, with the animation slot implementation I did, these animation blueprints can remain templates, which are not coupled to any specific skeleton assets, thus reusable.



Requirement 2:

2 - Link the locomotion anim layer instance at runtime by hitting a key to demonstrate multiple locomotion sets working on the same main animation blueprint.

The linking of the layer is done in C++ on IA_Swim action:

```
void AFCTBaseCharacter::IA_Swim(const FInputActionValue& InValue)
{
    if (LocomotionTypeData.IsEmpty())
    {
        return;
    }

    ELocomotionTypes TargetLocoType = ELocomotionTypes::Swimming;
    if (CharInfo->CurrentLocoType == ELocomotionTypes::Swimming)
    {
        TargetLocoType = ELocomotionTypes::Grounded;
    }

    TSubclassOf<UFCTBaseAnimInstance>* AnimClass = LocomotionTypeData.Find(TargetLocoType);
    if (!AnimClass)
    {
        return;
    }

    CharInfo->CurrentLocoType = TargetLocoType;

    GetMesh()->LinkAnimClassLayers(*AnimClass);

    if (CharInfo->CurrentLocoType == ELocomotionTypes::Swimming)
    {
        GetCharacterMovement()->SetMovementMode(MOVE_Flying);
        BaseAnimInstance->SetIgnoreRootMotionMode();
    }
    else
    {
        GetCharacterMovement()->SetMovementMode(MOVE_Walking);
        BaseAnimInstance->RestoreRootMotionMode();
    }

    BaseAnimInstance->UpdateAnimationSlots();
}
```

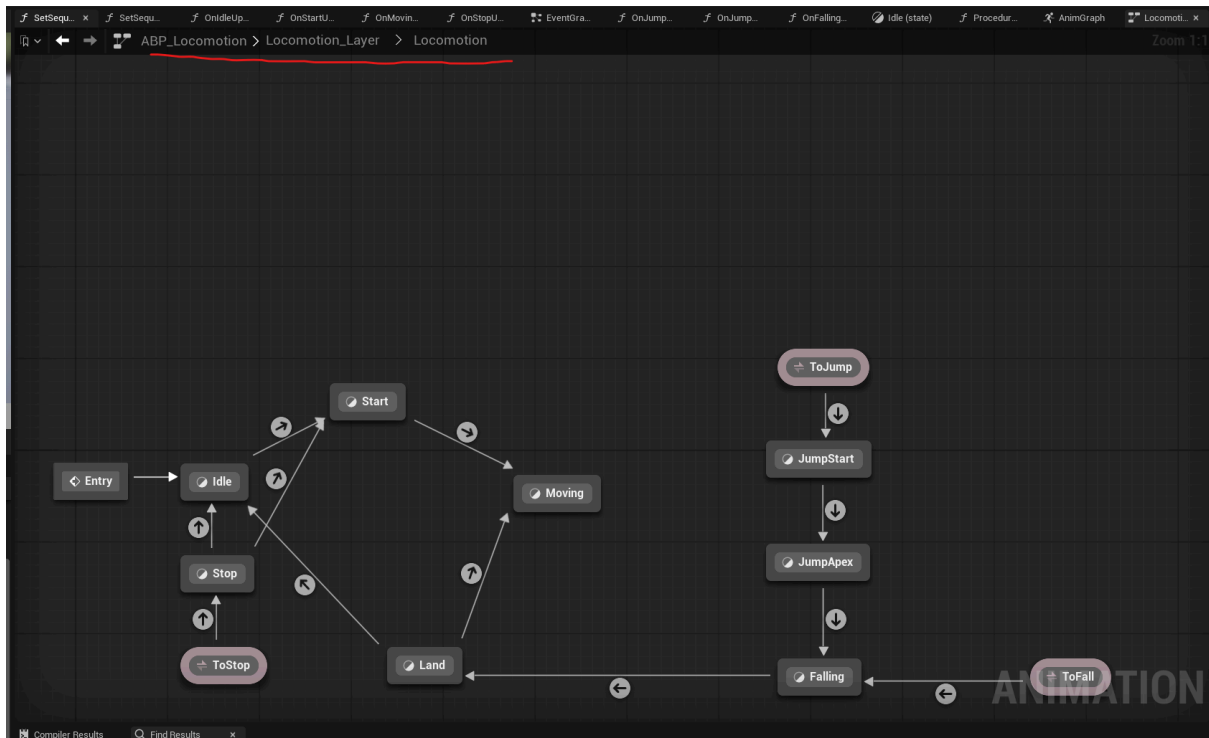
During gameplay, at the press of a key, we switch linked layers, from ABP_Locomotion to ABP_Locomotion_Swimming and back.

Both make use of the same animation slot system to load animation assets, but regardless, two different set of locomotion logic are switched at runtime.

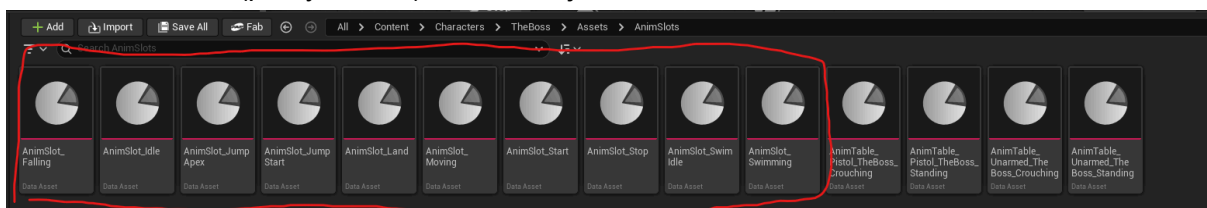
Requirement 3:

3 - Your Locomotion Animations should include walking, possible crouched variations, and jumping/falling, with appropriate transitions between states.

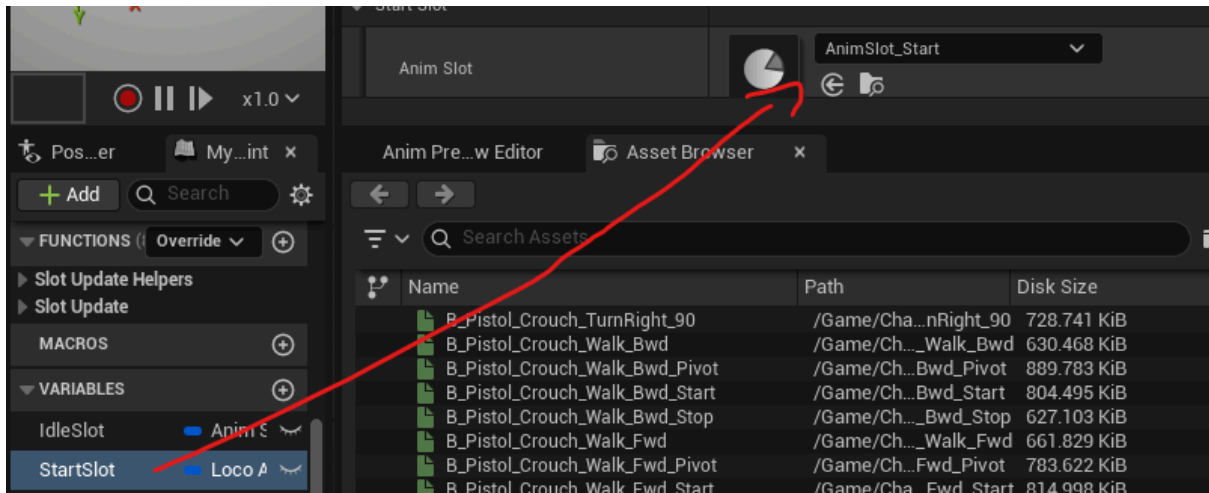
For this requirement, instead of following the same logic as lyra, meaning, having crouching, jog/walk, jumping/falling implemented by animation layers with specific data for each type, I only have one animation layer that is able to switch between all these sets. I say one, but both ABP_Locomotion and ABP_Locomotion_Swimming have this ability.



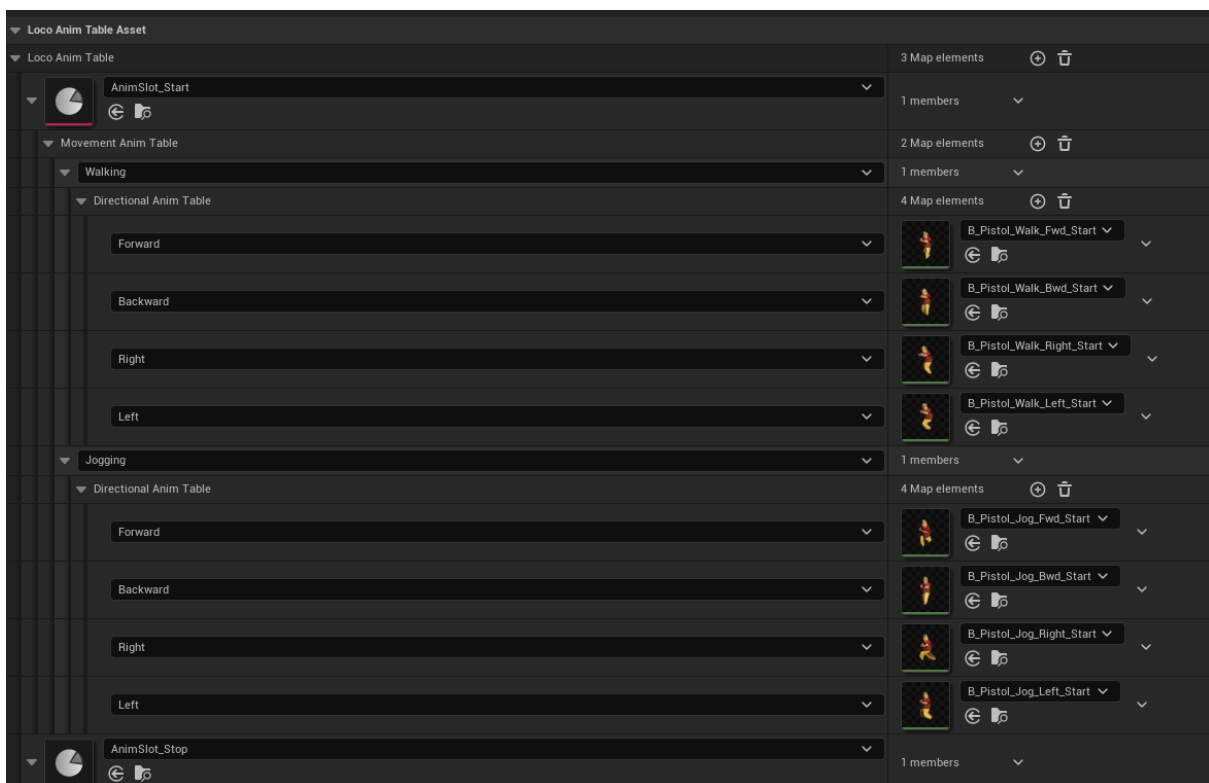
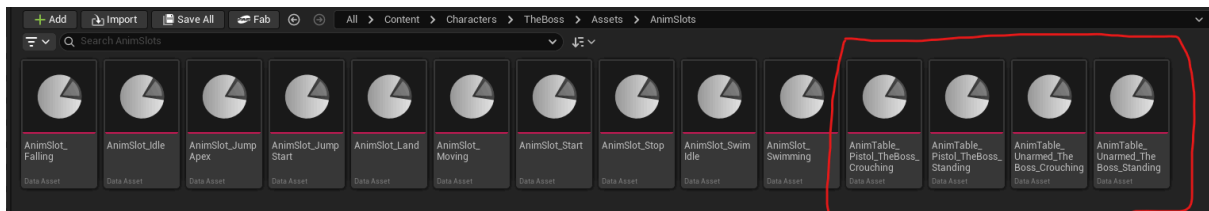
Again, the inspiration was proxy assets and tables, so for this I've implemented the concept of animation slots (proxy assets), these are just IDs in file format.



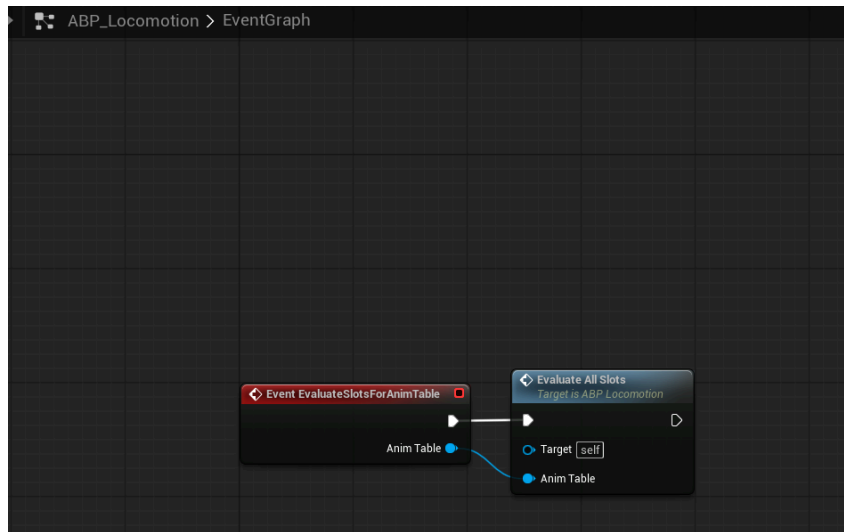
These are then set up in the blueprint as variables pointing to the asset.



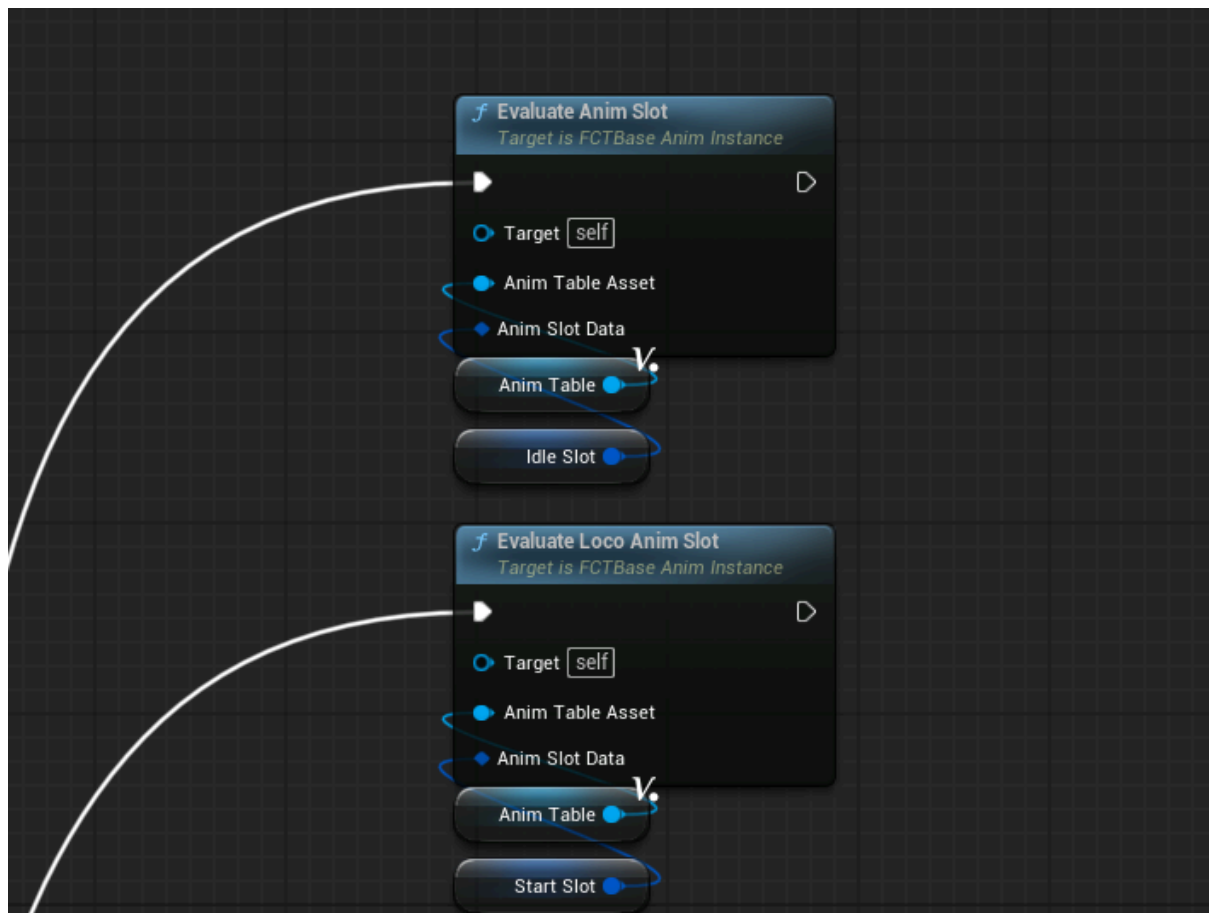
These are also mapped in UDataAssets called AnimTables (Proxy Tables):



As an example let's look closer at ABP_Locomotion. This abp inherits from FCTAnimInstanceBase which implements functionality to load the abp slots with actual animation sequences.

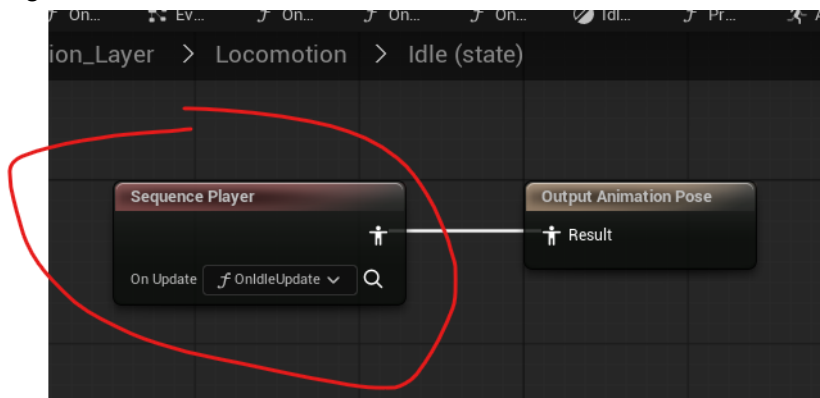


This function is a BlueprintNativeEvent and gets called through C++ only, specifically by gameplay when we switch animation tables. Then someone working on the blueprint is responsible for loading the slots, this is because the slots are defined in the blueprint and C++ does not know about them, so EvaluateAllSlots is a blueprint function that goes through all slots and calls another UFCTAnimInstanceBase implemented function:

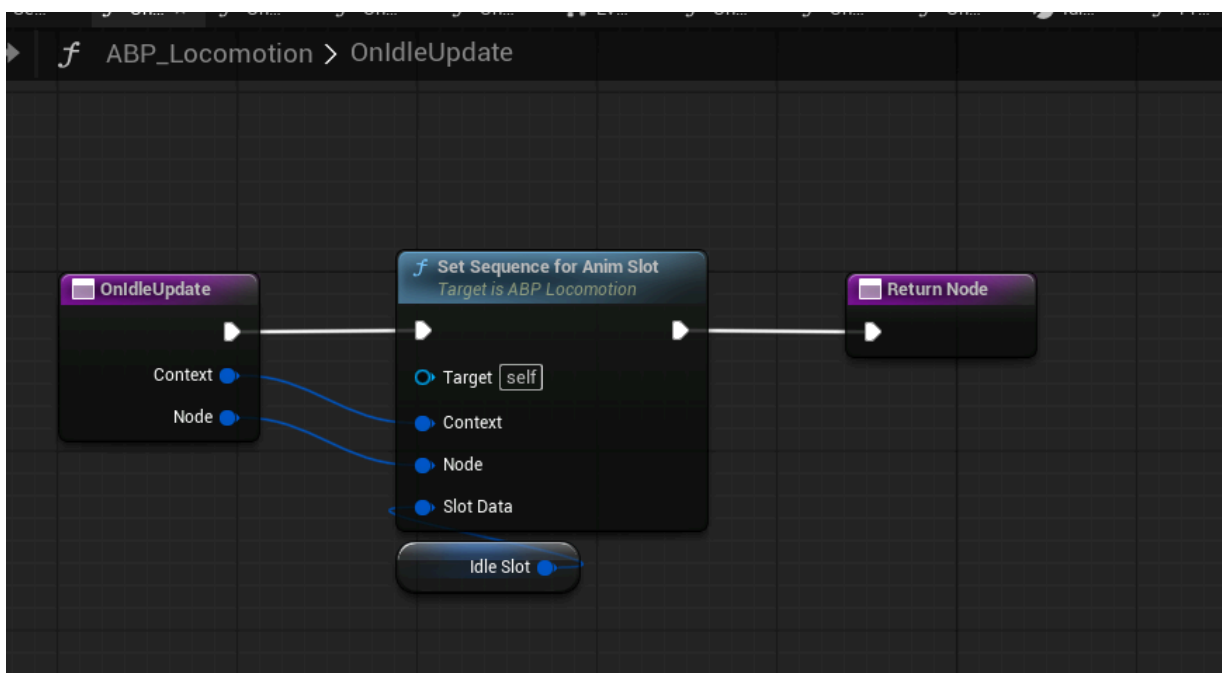
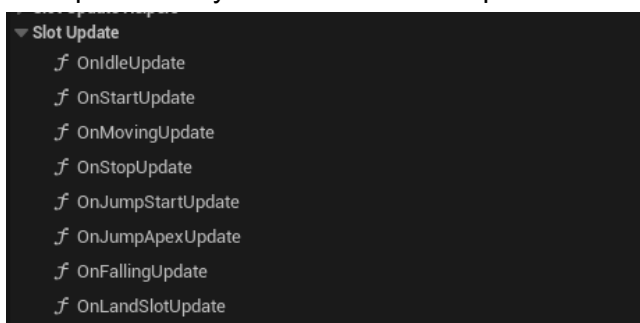


Or two functions, they're similar in regards to loading the correct animations. EvaluateAnimSlot is a generic one anim to slot, while EvaluateLocoAnimSlot is able to select an animation from a gait and direction set. I would have liked to add more flexibility at this point in the blueprint to allow more decision in the blueprint logic, but with the limited time I decided to do it in C++ internally. Each slot is filled with one animation sequence reference that will be used within the state machine.

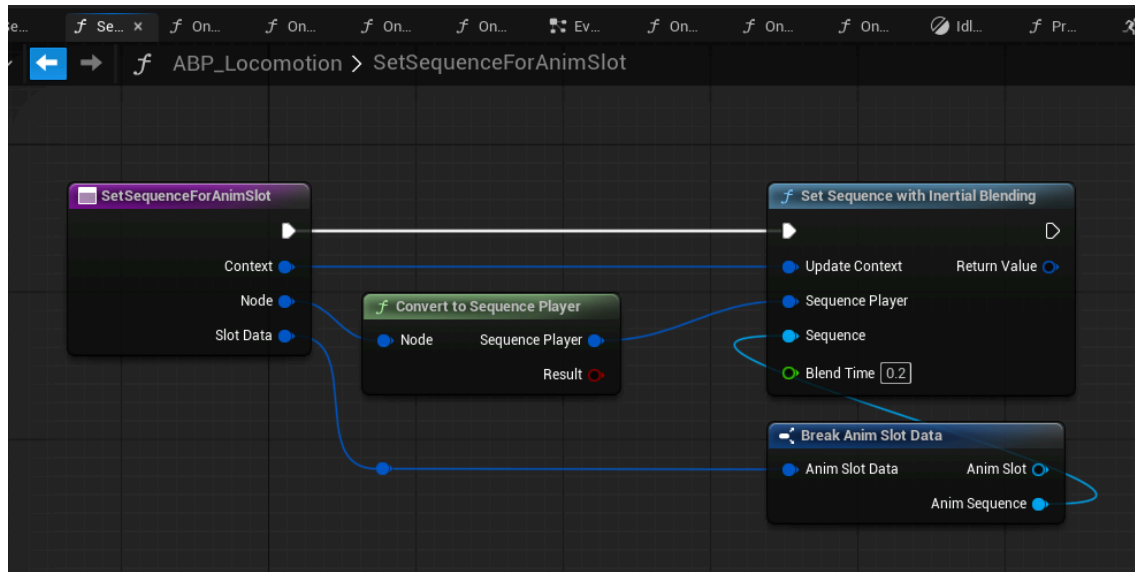
Going back to the state machine, a state is very simple, and all of them share the same logic:



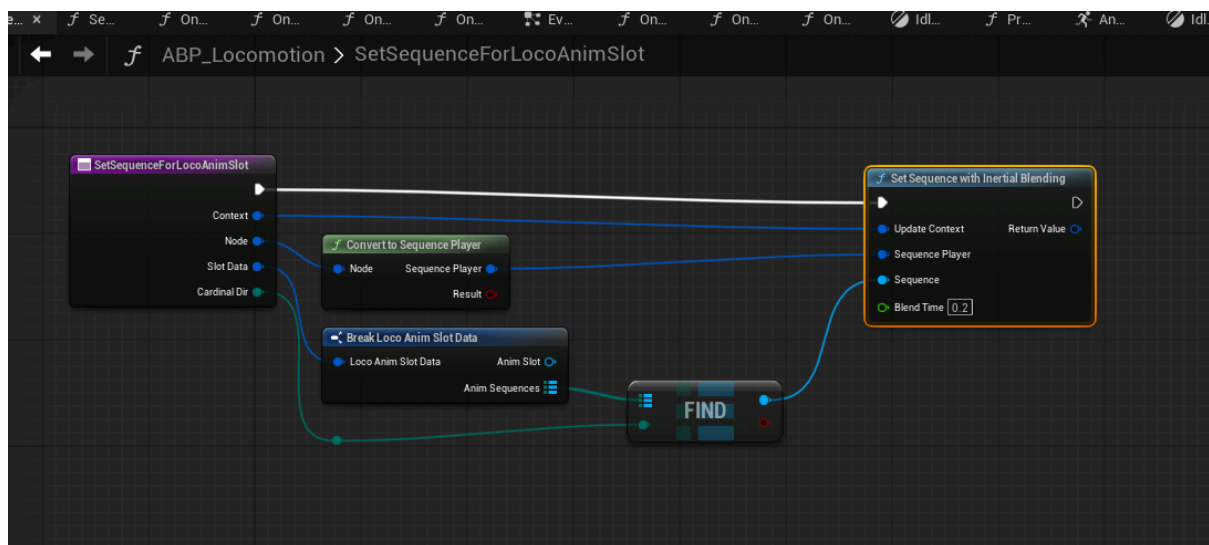
A Sequence Player is used with OnUpdate function binding where the asset is actually read:



SetSequenceForAnimSlot is a blueprint implemented function that leverages SetSequenceWithInertialBlending node:

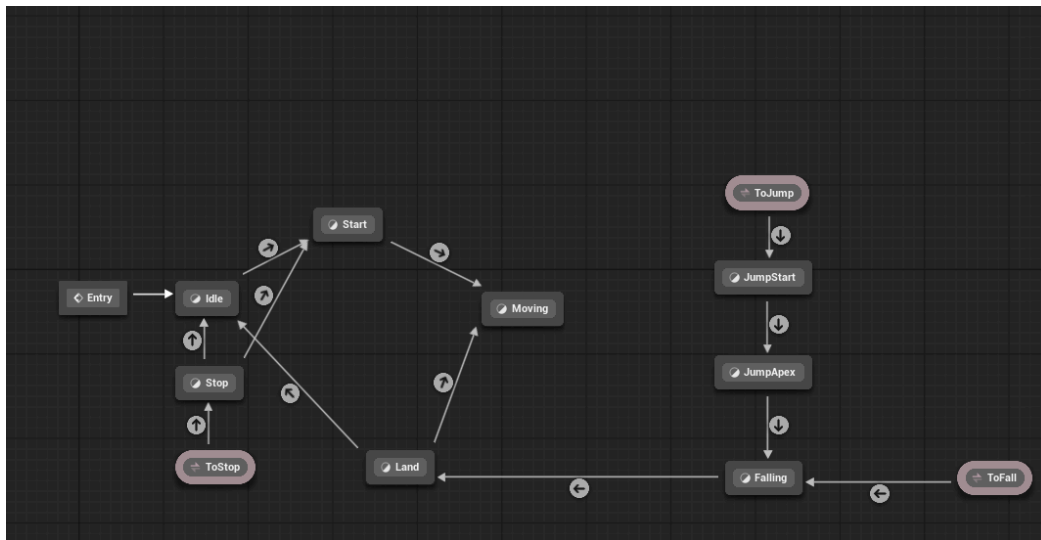


Locomotion Animation Slots have their own implementation:



SeSequenceWithInertialBlending is where all the magic happens, it keeps track of the animation sequence internally so it can blend between them. So when an animation table changes, the node will detect that and start an interiorization blend which besides being cheaper than traditional cross-fade, it's good to blend between locomotion animation sets due to momentum preservation from the bone velocities.

So going back to the requirement, this character can jog and walk, standing/crouching both unarmed and with a pistol. Besides that it can also jump with different unarmed and pistol animations and transitions between these sets with interiorization and a mix of cross-fading and inertialization for the state machine transitions.

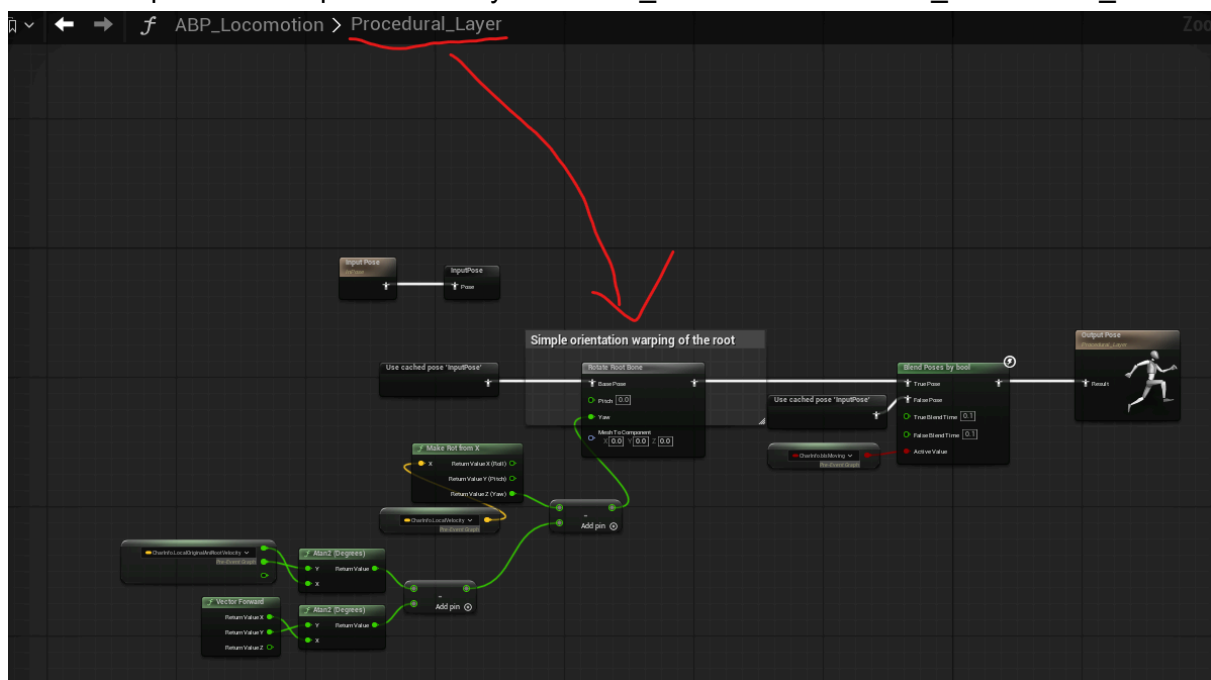


Requirement 4:

4 - Feel free to use any of the sample animations provided by Unreal, as well as any other freely obtained assets (e.g. Mixamo).

Requirement 4 is not really a requirement, so I will use this to talk about some more bonus I did:

I've also implemented a procedural layer for ABP_Locomotion and ABP_Locomotion_Swim.



This one rotates the root bone to align the mesh to root motion velocity, which at this point was altered so we can move in diagonals.

The animations only provide forward, backwards, right and left root motion, but I still wanted the character to move diagonally per player input, so I had to override the character movement component to alter velocity after root motion is applied.

This does not warp the root motion per say, but with limited time this was the quickest approach. I've added a comment with some solutions I've seen in my previous experiences, such as a custom FRootMotionSource to rotate the animation root motion towards a direction.

```

/*
 * This function is the easiest way to steer the character with root motion towards player input, it overrides velocity after root motion applies to it.
 * A better approach might be using ProcessRootMotionPreConvertToWorld to actually rotate the extract root motion before
 * its converted into world space, or creating custom FRootMotionSource that will adjust the extract root motion every turn.
 */
void UFCTCharacterMovementComponent::ApplyRootMotionToVelocity(float deltaTime)
{
    Super::ApplyRootMotionToVelocity(deltaTime);

    if (HasAnimRootMotion())
    {
        OriginalRootMotionVelocity = Velocity;

        float CurrentSpeed = Velocity.Size2D();
        if (CurrentSpeed < SMALL_NUMBER)
        {
            return;
        }

        FVector InputDir = GetLastInputVector().GetSafeNormal();
        if (InputDir.Size2D() < SMALL_NUMBER)
        {
            InputDir = PreviousInput;
        }

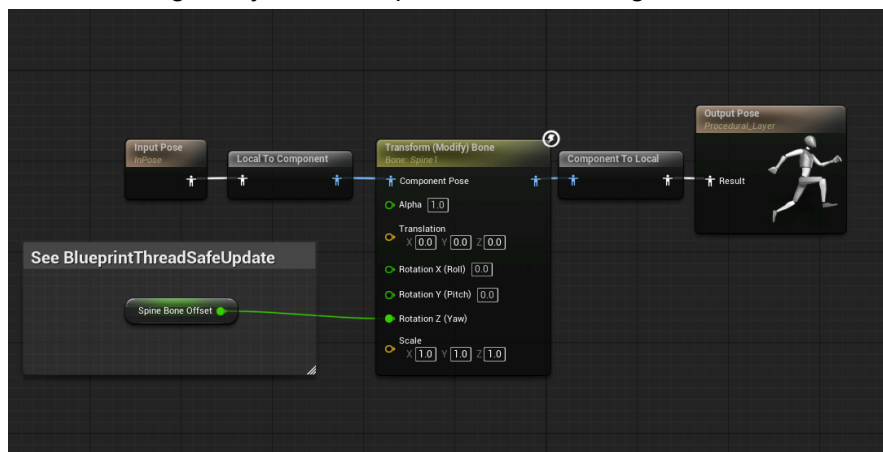
        FVector VelocityOverride = InputDir * CurrentSpeed;
        Velocity.X = VelocityOverride.X;
        Velocity.Y = VelocityOverride.Y;

        PreviousInput = InputDir;
    }
    else
    {
        OriginalRootMotionVelocity = FVector::ZeroVector;
    }
}

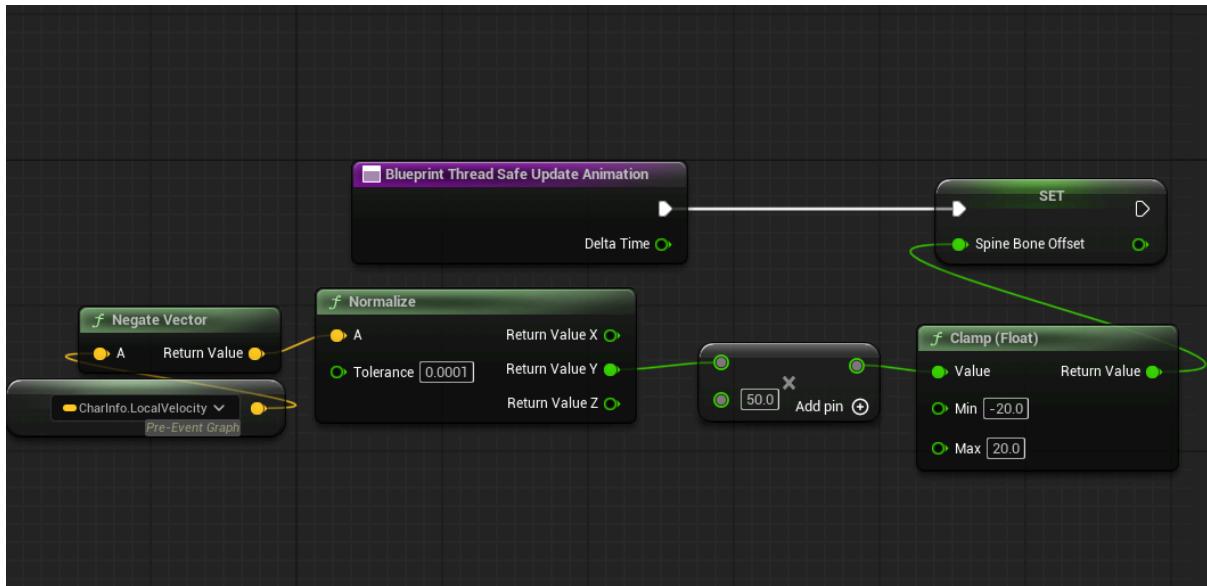
```

Then to more or less align the mesh to the new velocity I apply a Rotate Root Bone towards local velocity rotated by the original root motion velocity. This is because the root bone in the animation is always pointing forward regardless of movement direction, so by getting the angle difference between unit x and original root motion we get 90/-90 when moving sideways or 180 when moving backwards, which is then removed from the local velocity yaw, giving us the amount the root bone needs to turn to match the local velocity while keeping the animation quality (somewhat).

For swimming I only turn the spine bone left or right based off local velocity direction directly.



I've also used BlueprintThreadSafeUpdateAnimation which should be used instead of the EventGraph if one must keep calculations in the blueprint.



Since I lacked time, I kept the logic here and the ABP_Locomotion one directly in the anim graph, but I would move those to code, and since they related to animation only I would keep the calculation in the UFCTAnimInstanceBase.